Loop Rolling Algorithm

Detecting and collapsing repeating statement patterns into for-loops

2026-03-05

Overview

Loop rolling is a **pure str-to-str Python AST pass** that detects repeating statement patterns in fully-unrolled tiled functions and replaces them with `for` loops. It iterates until convergence to produce maximally nested loop structures.

Input: Python source code with a function definition containing unrolled tile computations. **Output:** Equivalent source code with repeating blocks collapsed into loops.

Example

Before:

```
tensor_0 = nl.ndarray((128, 128), ...)
nisa.dma_copy(dst=tensor_0[0:128, 0:128], src=lhs[0:128, 0:128])
tensor_1 = nl.ndarray((128, 128), ...)
nisa.dma_copy(dst=tensor_1[0:128, 0:128], src=lhs[128:256, 0:128])
tensor_2 = nl.ndarray((128, 128), ...)
nisa.dma_copy(dst=tensor_2[0:128, 0:128], src=lhs[256:384, 0:128])
```

After:

```
for i_0 in range(3):
    tensor_0 = nl.ndarray((128, 128), ...)
    nisa.dma_copy(dst=tensor_0[0:128, 0:128],
                  src=lhs[i_0 * 128:(i_0 + 1) * 128, 0:128])
```

Architecture

The algorithm has three layers. **`roll_loops`** calls `_roll_once` in a fixpoint loop until no more patterns exist. Each `_roll_once` parses the source, locates the `FunctionDef`, assigns the next loop variable name (`i_0`, `i_1`, ...), and delegates to `_try_roll_in_body`.

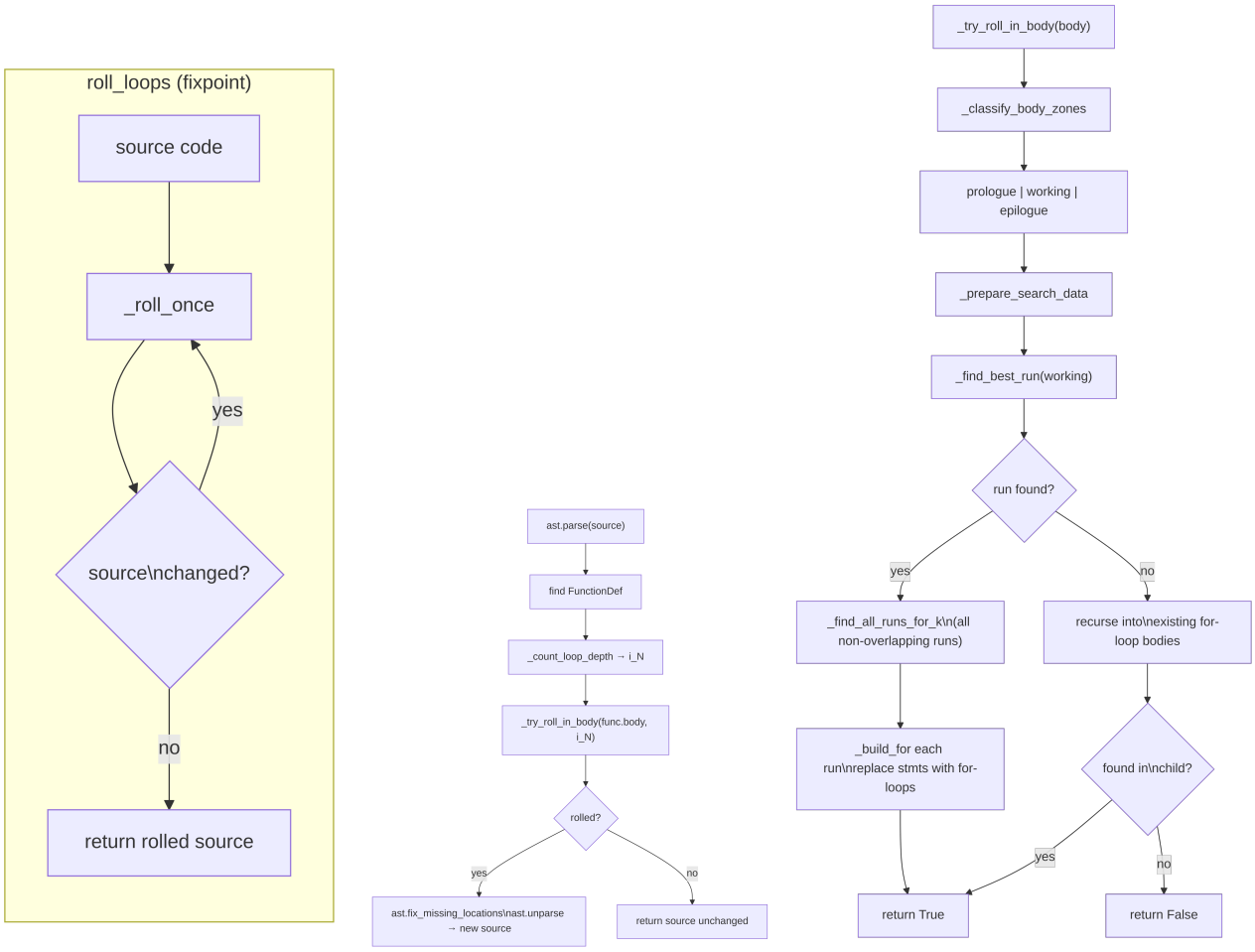

Figure 1: Left: fixpoint loop. Center: single rolling step. Right: recursive body search with shared precomputation.

`_try_roll_in_body` calls **`_prepare_search_data`** once to build all precomputed data structures, then searches for the best repeating run. When a run is found, **`_find_all_runs_for_k`** collects all non-overlapping runs at the same block size, and each is replaced by a for-loop. If no patterns are found at the current level, the algorithm recurses depth-first into existing for-loop bodies (first match wins).

Core Algorithm: `_find_best_run`

Given N working statements, find the repeating run with maximum **coverage** $= K \times C$, where K is block size and C is trip count. The algorithm applies five validation stages in sequence: fingerprint prefilter, external-name prefilter, structural normalization, arithmetic progression check, and scope safety check.

Stage	Filter	Cost	Rejection Rate
1. Fingerprint	Hash match (names/ints erased)	$O(1)$ per stmt (numpy vectorized)	Coarse
2. External names	Referenced − assigned set comparison	$O(K)$ set unions	97–100% of stage 1 survivors

Stage	Filter	Cost	Rejection Rate
3. Normalization	Positional rename + structural compare	$O(K \cdot L)$ regex	Exact structural match
4. AP check	Integer constants form arithmetic progressions	$O(K \cdot C)$ DFS	Rejects non-linear patterns
5. Scope safety	No variable leaks out of rolled region	$O(1)$ precomputed	Count-retry on failure

Each stage filters candidates before the next, more expensive check.

Search Space

The algorithm exhaustively scans all block sizes $K \in [1, N/2]$ and all starting positions $P \in [0, N-2K]$. **Pruning:** block size K is skipped early when its theoretical maximum coverage $K \times \lfloor N/K \rfloor$ cannot beat the current best.

Step 1: Fingerprint Prefilter

Before expensive normalization, each statement is fingerprinted by hashing its `ast.dump` with all names erased to `_` and all integer constants replaced with `0`. Two blocks can only match if their per-statement fingerprint sequences are identical. Fingerprint matching across all positions for a given K uses numpy vectorized strided comparison (`_block_match_positions`), completing in $\sim$0.3ms per K.

Step 2: External-Name Prefilter

For blocks that pass the fingerprint check, a cheap prefilter compares the **external name sets** (referenced names minus assigned names) of the reference block and the candidate block. Per-statement assigned and referenced name sets are precomputed once via a single `ast.walk` pass, so computing block-level external names is just $O(K)$ set unions. Two blocks can only match after normalization if they have identical external names, since normalization preserves external names verbatim. This filter rejects 97–100% of false positives from the fingerprint stage, eliminating expensive normalization calls for structurally non-matching blocks.

Step 3: Structural Normalization

For blocks that pass both prefilters, full normalization is applied:

1. **Collect assignment targets** in the block and map each to a positional name (`_v0`, `_v1`, ...) in order of first appearance.
2. **Look up pre-zeroed dump strings** for each statement (integer constants already replaced with `0` during the precomputation phase).
3. **Rename** all local variable names in the joined dump string using the positional map. External variables (defined outside the block) are left as-is.

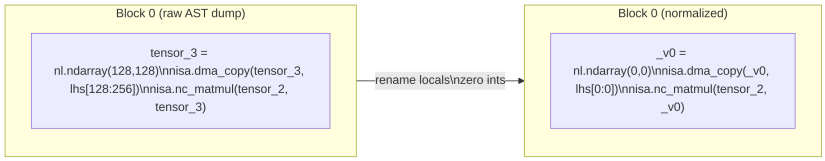

Figure 2: Normalization renames local variables to positional names and zeros integer constants.

Two blocks are **structurally identical** when their normalized strings match: same AST shape, same external references, differing only in local names and integer constants.

Step 4: Arithmetic Progression Check (`_extract_varying`)

For each integer constant position (identified by DFS path in the AST), collect its value across all C blocks and verify it forms an arithmetic progression:

$$v_i = \text{base} + i \times \text{stride}, \quad i = 0, 1, \ldots, C-1$$

Constants with stride $= 0$ are invariant across blocks. Constants with stride $\neq 0$ are **varying** and become expressions of the loop variable. A candidate is **invalid** if any constant fails the AP check (e.g., `[0, 128, 0]` when tiles wrap around a 2D grid).

Step 5: Scope Safety Check

Verify that no variable **defined** inside the rolled region is **referenced** after the rolled region:

$$\text{defined}(\text{rolled_region}) \cap \text{referenced}(\text{after_region}) = \emptyset$$

This prevents rolling from hiding a definition that later code depends on. When a variable is assigned inside a loop, only the last iteration's value survives; if code after the loop expects a specific earlier iteration's value, rolling would silently corrupt the computation.

When the scope check fails for count C, the algorithm decrements to $C - 1$ and retries, continuing down to count $= 2$. This recovers shorter valid sub-runs (e.g., 5 out of 7 blocks when only the last 2 have cross-references).

Code Generation: `_build_for`

Once a valid `_LoopRun(start_idx, block_size, trip_count, varying)` is found:

1. **Deep-copy** the first block as a template.
2. Replace each varying constant's AST node with an expression of the loop variable:

Condition	Generated Expression
stride $= 0$	`base` (literal constant)
base $= 0$	`i * stride`
base mod stride $= 0$	`(i + base/stride) * stride`
otherwise	`i * stride + base`

3. Wrap in `ast.For(target=i_N, iter=range(trip_count), body=template)`.
4. Splice the for-loop into the body, replacing the original $K \times C$ statements.

Zone Classification

Before searching, the function body is partitioned into three zones:

Zone	Detection	Treatment
Prologue	Leading `np.empty(...)` allocations	Excluded from search
Working	Everything between prologue and epilogue	Searched for patterns
Epilogue	Trailing `return` statement	Excluded from search

Only the working zone is passed to `_find_best_run`. The prologue exclusion prevents allocation statements from being rolled into loops.

Complexity

For N working statements:

- **Precomputation:** $O(N)$ `ast.dump` calls (cached), $O(N)$ `ast.walk` for per-statement name sets and suffix reference unions (single merged pass), $O(N)$ regex substitutions for pre-zeroed dump strings.
- **Outer loop:** $O(N/2)$ block sizes, each with a numpy vectorized fingerprint match (`_block_match_positions`).
- **External-name filter:** $O(K)$ set unions per candidate position. Rejects 97–100% of fingerprint false positives, preventing expensive normalization.
- **Normalization:** $O(K \cdot L)$ where L is average dump string length. Only reached for positions passing both fingerprint and external-name checks.
- **Total:** $O(N^2)$ worst case per `_roll_once` call, but effective runtime is much lower due to the multi-stage filtering pipeline. The fixpoint loop calls `_roll_once` up to $O(\log N)$ times (each call reduces statement count).

Performance

Benchmarked on a 2801-statement unrolled NKI kernel (64 output tiles from an 8×8 tiled 1024×1024 matmul). The rolling pass produces 553 lines with 42 for-loops across 17 fixpoint iterations.

Optimization Progression

Optimization	Wall Time	Speedup
Baseline (fingerprint prefilter only)	21.83 s	1.0×
+ External-name prefilter, pre-zeroed dumps	7.24 s	3.0×
+ Merged `ast.walk` pass	6.45 s	3.4×
+ Shared precomputation across search calls	5.86 s	3.7×

The external-name prefilter provides the largest single improvement by eliminating 97–100% of false-positive fingerprint matches before expensive normalization. Pre-zeroed dump strings remove a redundant regex pass. Merging the `ast.walk` pass and sharing precomputed data between `_find_best_run` and `_find_all_runs_for_k` eliminate duplicate work.

Per-Iteration Profile

Most work happens in the first few iterations when the statement count is highest. Later iterations process smaller bodies and complete quickly.

Iteration	Time (ms)	Output Lines	Cumulative Loops
0	1602	2500	1
1	1059	2237	2
2	638	2062	3
3	461	1582	7
4	255	1233	23
5–17	<200 each	553 (final)	42 (final)

Performance Floor

The numpy `_block_match_positions` calls across all $\sim N/2$ block sizes take $\sim$500ms per iteration at $N = 2800$. This is a fixed cost that cannot be reduced by filtering, since every block size must be checked to find the globally optimal run. Further speedup would require algorithmic changes such as period detection to prune block sizes.

Known Limitations

1. **Cross-scope references from data reuse.** When data reuse transforms share loads between tiles, the NKI lowering emits a variable definition inside one tile and a reference from another tile. This creates a cross-tile dependency that fails the scope safety check for both the inner K-loop (the shared variable is inside the loop body) and the outer tile loop (the defining tile is inside, the referencing tile is outside).

2. **Non-uniform block sizes.** Data reuse transforms also produce tiles with fewer statements (e.g., 42 instead of 44), since shared loads are elided. Different-sized tiles cannot be part of the same run at the outer level.

3. **Prologue detection is framework-specific.** `_is_alloc_stmt` only recognizes `np.empty(...)`. NKI code uses `nl.ndarray(...)`, which is not detected as prologue. Benign in practice (the output allocation simply becomes part of the working zone).

4. **Unparse–reparse normalization.** The `ast.unparse` → `ast.parse` cycle between fixpoint iterations is essential: it normalizes AST expressions (e.g., collapsing `0 + x` to `x`) so that subsequent iterations discover optimal patterns. Removing this cycle (working directly on the AST) converges to a worse fixpoint (611 lines vs 553).